Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 6

Lists and nested lists

Not graded, not handed in. Do them anyway. This is where the learning happens! Solutions are published after we discuss them in class.

Predict before you run. Every question in Part A has a definite answer you can work out on paper. Write it down first, then check it in Python. The questions you get wrong on paper are the ones worth understanding.

Part A — The list type

A1 *Follow the list*

Write down `t` after each numbered line, and the values of `x` and `y` once they exist.

```
t = [5, 3, 8]
t.append(3)          # 1
t.insert(0, 9)       # 2
x = t.pop(0)         # 3
t.remove(3)          # 4
y = t.pop(1)         # 5
t.append(x + y)      # 6
```

Then: what would `t.remove(4)` do at the end?

A2 *Which lines lose the list?*

Each line below starts from a fresh `g = [3, 1, 2]`.

```
1.  g = g.sort()
2.  g = sorted(g)
3.  g.append(4)
4.  g = g.append(4)
5.  g = g + [4]
6.  g = g.reverse()
```

a) Three of them leave `g` as `None`. Which?

b) For the other three, what is `g` afterwards?

c) State the rule in one sentence.

A3 *Draw it*

For each program, draw a box for every list and an arrow from every name to the box it points at, as it stands at the end. Then say what the last line prints.

```
# (a)                  # (b)                  # (c)
a = [1, 2]             a = [1, 2]             a = [1, 2]
b = a                  b = a                  b = a[:]
a.append(3)            a = a + [3]            b[0] = 99
print(b)               print(b)               print(a)
```

(a) and (b) differ in one line and print different things. Explain the difference in one sentence.

A4 *The copy that is not a copy*

This is supposed to add two points to every grade, capped at 30, and keep the original grades for comparison.

```
grades = [28, 30, 24, 15, 27]
curved = grades

for i in range(len(curved)):
    curved[i] = curved[i] + 2
    if curved[i] > 30:
        curved[i] = 30

print("before:", grades)
print("after: ", curved)
```

a) What does it print?

b) Which line is wrong, and why?

c) Fix it by changing that one line.

A5 *First come, first served*

A lab has a waiting list. Students signed up by typing their name, and some signed up more than once.

```
signups = ["Linus", "Ada", "Linus", "Grace", "Ada", "Alan", "Ada"]
```

Build a list with each name once, in the order in which the students *first* signed up.

a) Write the comment plan first.

b) Build the list with a loop, `in` and `append`. For these names it should be `["Linus", "Ada", "Grace", "Alan"]`.

c) Print how many of the sign-ups were duplicates.

d) Someone suggests tidying the result with `sorted`. Why is that wrong here?

Part B — Iterating over lists

B1 *Six expressions on a table*

```
t = [[3, 8, 1],
     [6, 2, 9],
     [4, 7, 5]]
```

Give the value of each.

```
1.  t[0]            4.  t[-1][-1]
2.  t[2][0]         5.  len(t[1])
3.  t[0][2]         6.  t[1][1:]
```

Then write the expression that gives `9`, and the one that gives the item in the middle of the table.

B2 *It tries, and it cannot*

```
names = ["ada", "alan", "grace"]

for name in names:
    name = name.title()

print(names)
```

a) What does it print?

b) Why? Two sentences: one about the loop, one about strings.

c) Fix it with a loop over positions.

d) Fix it again, in one line, with a comprehension.

B3 *Comprehensions, both ways*

Throughout: `words = "the quick brown fox jumps over the lazy dog".split()`

Rewrite each loop as a single comprehension, and say what it produces.

```
# (a)
squares = []
for n in range(1, 6):
    squares.append(n * n)
```

```
# (b)
short = []
for w in words:
    if len(w) <= 3:
        short.append(w)
```

```
# (c)
loud = []
for w in words:
    loud.append(w.upper())
```

Now the other way round. Rewrite this as a loop with an accumulator, and say what it produces:

```
lengths = [len(w) for w in words if w != "the"]
```

B4 *A table of scores*

Four students, three exams. Each row is one student.

```
scores = [[28, 24, 30],
          [18, 27, 25],
          [30, 30, 29],
          [21, 19, 26]]
```

a) Print each student's total.

b) Print the mean of each exam. This one is harder: a column is not a list you can loop over directly.

c) Build a new table `flipped` with the rows and columns swapped — three rows of four — so that `flipped[c][r]` is `scores[r][c]`.

Your code should still work if a fifth student or a fourth exam is added. No number 4 or 3 anywhere.

B5 *Every position*

`xs.index(v)` only finds the *first* position of `v`. Write a program that builds a list of **all** the positions where `target` appears in `xs`, and prints it — or prints `not found` if there are none.

Check it with `xs = [3, 7, 3, 1, 3]` and targets `3`, `1` and `8`.

B6 *Second-largest, without sorting*

Find the second-largest value in a list with **one** pass through it: no `sort`, no `sorted`, no `max`. Assume the list has at least two items.

Hint: the running maximum from lecture 4, with two variables instead of one.

Test it on all of these, and write down what you get:

```
[28, 30, 24, 18, 27]    [30, 24, 28]    [-5, -2, -9]    [30, 30, 24]
```

Two of them are designed to break a first attempt. For the last one, decide what the answer *should* be, and say whether your program agrees.

B7 *A different way to sort*

This program sorts too, by a method called *insertion sort*. Each pass takes the next item and moves it left, one swap at a time, until the item to its left is not bigger.

```
xs = [6, 3, 9, 1, 4]
comparisons = 0

for i in range(1, len(xs)):
    j = i
    while j > 0:
        comparisons += 1
        if xs[j - 1] > xs[j]:
            tmp = xs[j - 1]
            xs[j - 1] = xs[j]
            xs[j] = tmp
            j -= 1
        else:
            break
    print(xs)

print(comparisons)
```

a) Trace it by hand. Write the list after each pass of the `for` loop.

b) How many comparisons does it make? Selection sort needs 10 for any five items.

c) Change the first line to the sorted list `[1, 3, 4, 6, 9]`. How many comparisons now? What does insertion sort notice that selection sort cannot?

d) Which list of the same five numbers makes insertion sort work hardest, and how many comparisons does it need then?

If you finish early

Predict what this prints. Then draw it, boxes and arrows, the way you did in A3.

```
a = [[1, 2], [3, 4]]
b = a[:]
b[0].append(9)
b[1] = [0]

print(a)
```

`b = a[:]` is a copy, and yet `a` changes. What exactly did `a[:]` copy, and what did it not?